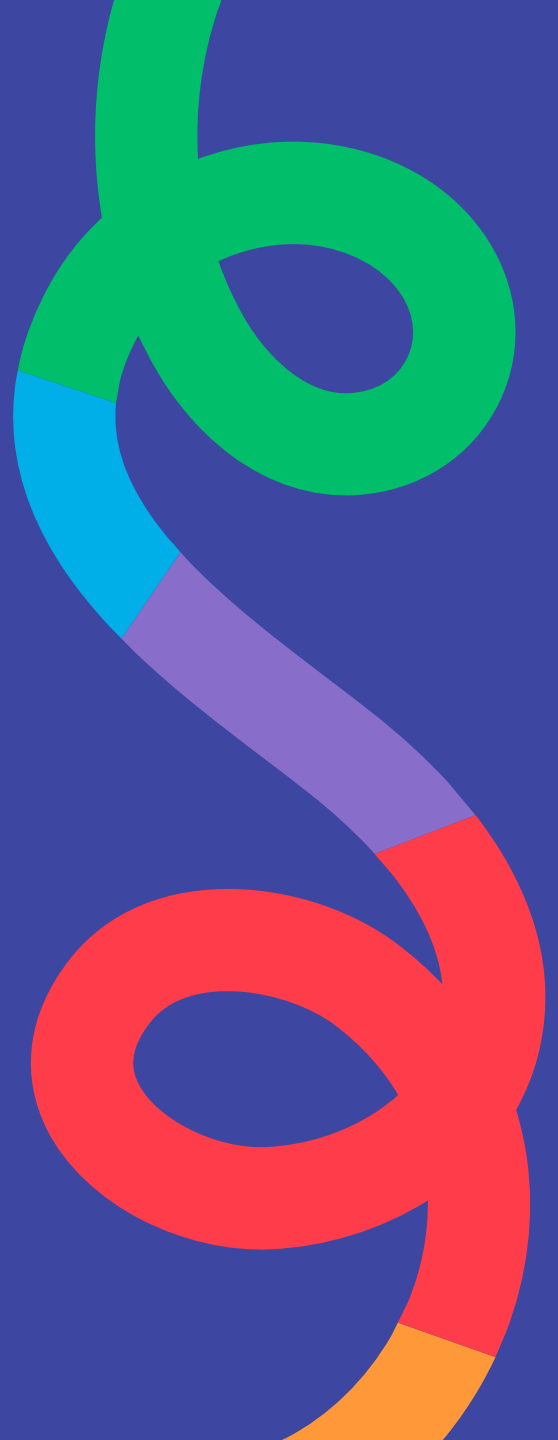
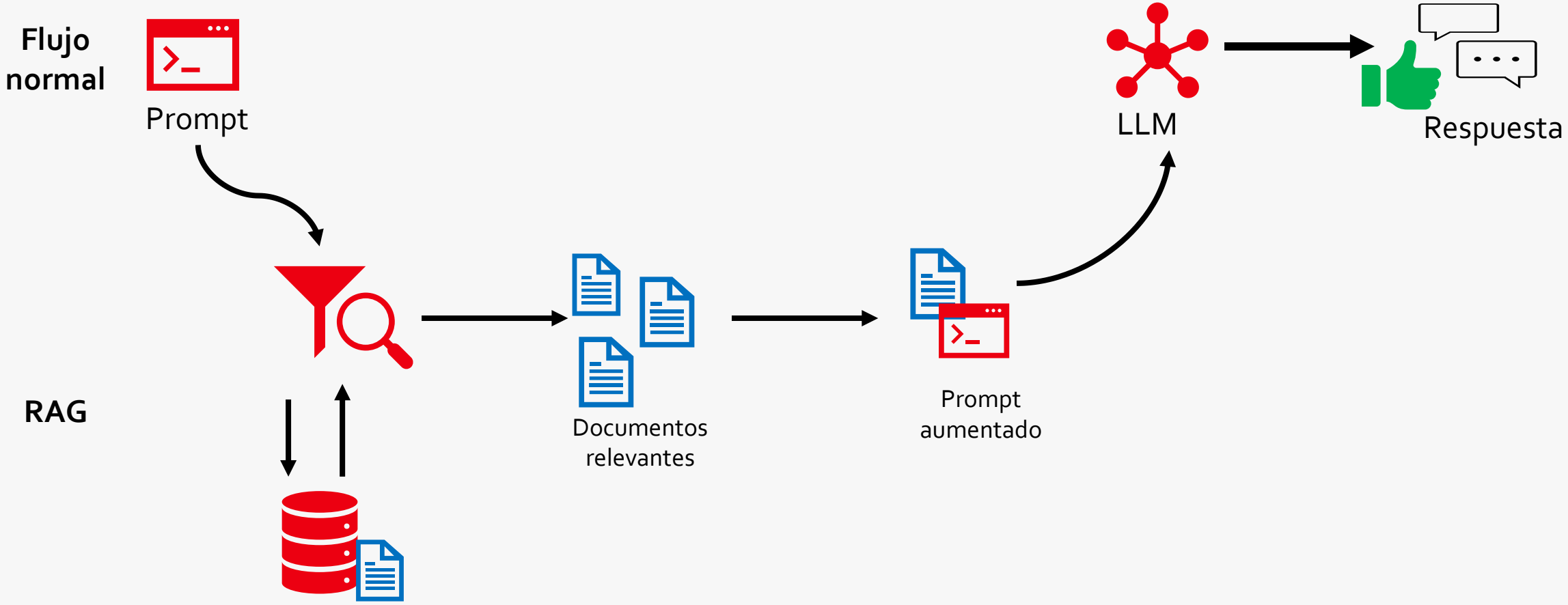
FUNDAMENTOS DE RECUPERACIÓN DE INFORMACIÓN

PUNTOS CLAVE DE LA AGENDA

1. Enfoques de búsqueda
2. Conceptos esenciales del modelo TF-IDF
3. Aplicación del mecanismo TF-IDF
4. Código de ejemplo



ARQUITECTURA RAG



DOS ENFOQUES DE BÚSQUEDA

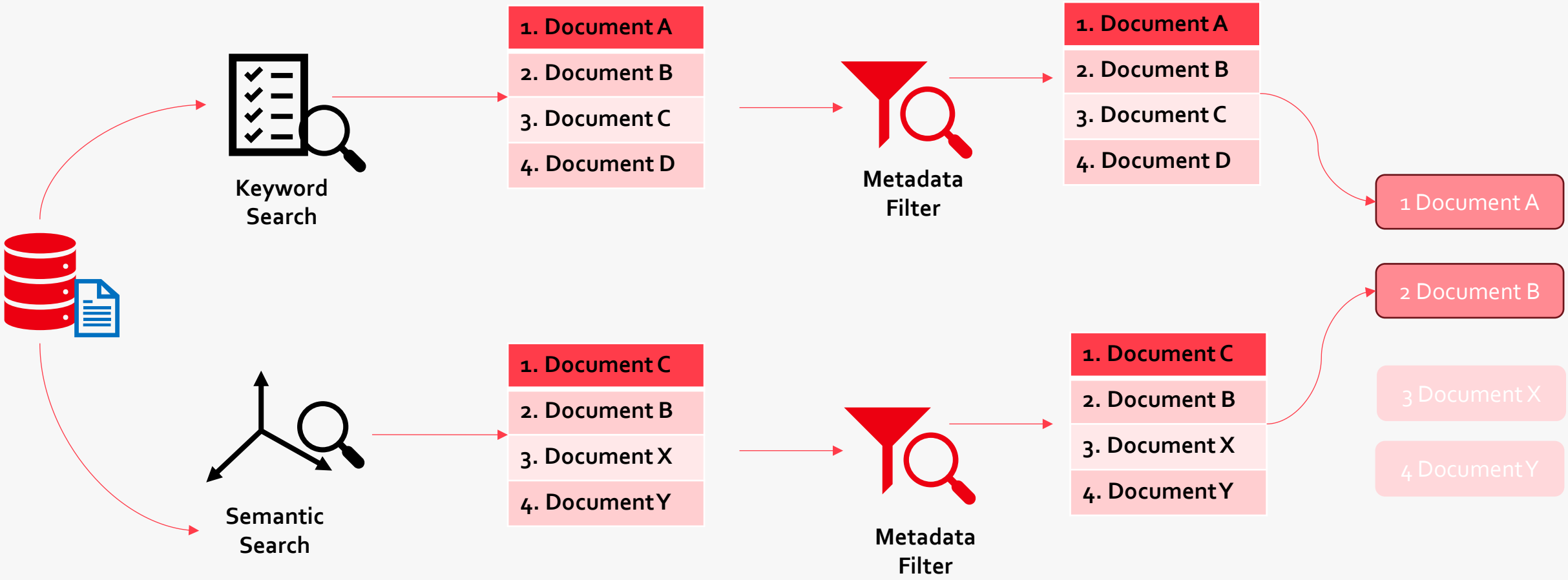
Keyword Search

- Busca documentos que contengan las palabras exactas que aparecen en el prompt

Semantic Search

- Busca documentos con un significado similar al enunciado.

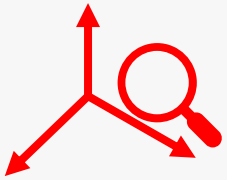
TÉCNICAS DE BÚSQUEDA



HYBRID SEARCH



Keyword Search
Garantiza sensibilidad a las palabras exactas el usuario incluido en el prompt



Semantic Search

Encuentra documentos con un significado similar, incluso sin palabras coincidentes



Metadata Filter

Excluye documentos basados en Criterios rígidos

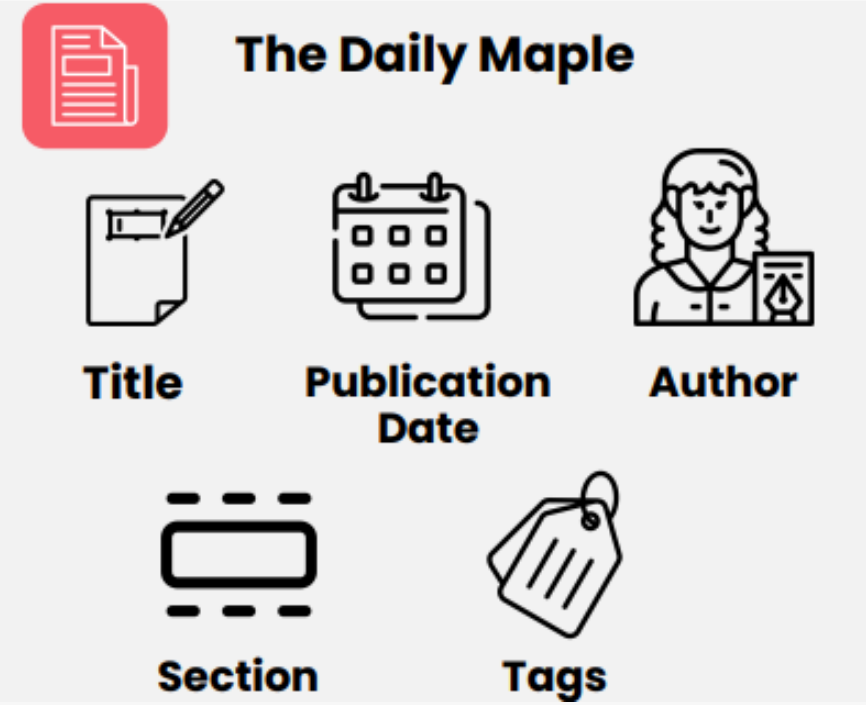


Los recuperadores de alto desempeño equilibran las tres técnicas según las necesidades del proyecto

METADATA FILTERING

METADATA FILTERING

Utiliza criterios rígidos para reducir documentos basándose en metadatos como título, autor, fecha de creación, privilegios de acceso y más.




```
SELECT * FROM articles
WHERE publication_date = '2023-10-01';
```

Return all articles published on specific date

```
section = "Opinion"
author = "Michael Chen"
date = June to July 2024
```

METADATA FILTERING



El filtrado de metadatos no realiza la recuperación, sino que reduce los resultados de otras técnicas basándose en los atributos del usuario, no en el contenido de la consulta.

Subscription Filtering



Filter: Exclude all articles with metadata "subscription = paid”

Geographic Filtering



Filter: Include only articles with metadata "region = North America”

VENTAJAS Y DESVENTAJAS

Ventajas

- Fácil de entender y depurar
- Rápido, optimizado, maduro y fiable
- Aplica reglas de recuperación estrictas, que coinciden exactamente con los criterios de filtrado

Desventajas

- No es un buscador real
- Rígido, ignora el contenido.
- Inútil si se utiliza solo



KEYWORD SEARCH

DEFINICIÓN Y PROPÓSITO DE KEYWORD SEARCH



Concepto de búsqueda por palabras clave

La búsqueda por palabras clave recupera documentos que contienen términos específicos de la consulta del usuario.

Propósito principal

Identificar información relevante en grandes colecciones textuales mediante términos visibles y explícitos.

Fundamento de motores de búsqueda

La búsqueda por palabras clave es la base para sistemas que permiten encontrar resultados útiles rápidamente.

TIPOS DE BÚSQUEDAS Y APLICACIONES COMUNES



Tipos de búsqueda por palabra clave

Existen búsqueda exacta, búsqueda booleana y búsqueda de frase para diferentes necesidades de consulta.

Aplicaciones comunes

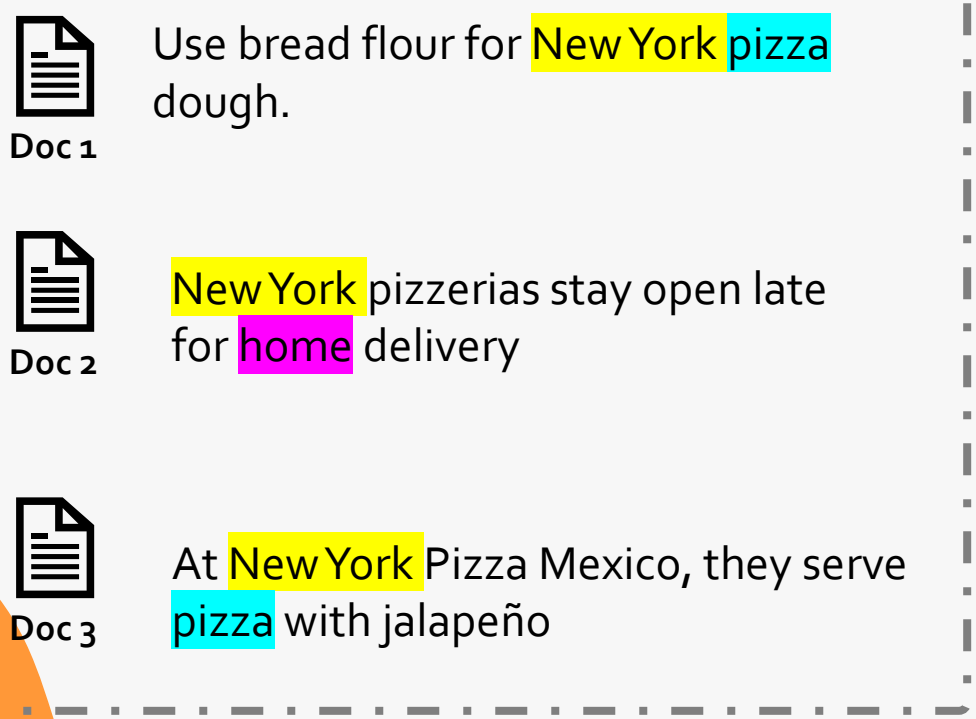
Estas búsquedas se aplican en motores web, bases de datos, sistemas legales, comerciales y comercio electrónico.

Búsqueda booleana avanzada

La búsqueda booleana usa operadores AND, OR y NOT para refinar resultados y consultas complejas.

FUNCIONAMIENTO

How can I make New York style pizza at home?





LIMITACIONES DE LOS SISTEMAS DE BÚSQUEDA SIMPLES

Limitaciones en comprensión de contexto

Los sistemas simples no entienden sinónimos ni polisemia, causando resultados irrelevantes o incompletos.

Falta de ponderación de palabras

No se mide la importancia relativa de palabras en documentos, afectando la relevancia de la información recuperada.



CONCEPTOS ESENCIALES DEL MODELO TF-IDF



SIGNIFICADO DE TERM FREQUENCY (TF)

Definición de TF

Term Frequency mide la cantidad de veces que una palabra aparece en un documento específico.

Normalización del Conteo

Normalizar el conteo frente a la longitud del documento evita sesgos por textos largos.

Importancia del TF

TF ayuda a identificar términos centrales en un documento para su análisis.

TERM FREQUENCY (TF)

Example Query: pizza oven

Document 1

Homemade pizza in oven is better than frozen pizza

Contains: Pizza (2x) Oven (1x)

Simple Scoring = 2 points

TF Scoring = 3 points

Document 2

Wood-fired oven is a better oven than a stone oven for cooking pizza

Contains: Pizza (1x) Oven (3x)

Simple Scoring = 2 points

TF Scoring = 4 points

TERM FREQUENCY (TF)

Example Query: pizza oven

Document 1

Homemade pizza in oven is better than frozen pizza

Contains: Pizza (2x) Oven (1x)

Simple Scoring = 2 points

TF Scoring = 3 points

Document 2

Wood-fired oven is a better oven than a stone oven for cooking pizza

Contains: Pizza (1x) Oven (3x)

Simple Scoring = 2 points

TF Scoring = 4 points

TERM FREQUENCY (TF) - NORMALIZACIÓN

Los documentos más largos pueden contener palabras clave muchas veces simplemente porque son más largos.

Solución: Normalizar por longitud del documento

Puntuación = (Número de ocurrencias de palabras clave) / (Total de palabras en el documento)

La puntuación básica de TF trata todas las palabras por igual, ya sean palabras de relleno comunes o términos raros y significativos.



SIGNIFICADO DE INVERSE DOCUMENT FREQUENCY (IDF)

Importancia global de palabras

IDF mide qué tan importante es una palabra en una colección completa de documentos, destacando términos únicos.

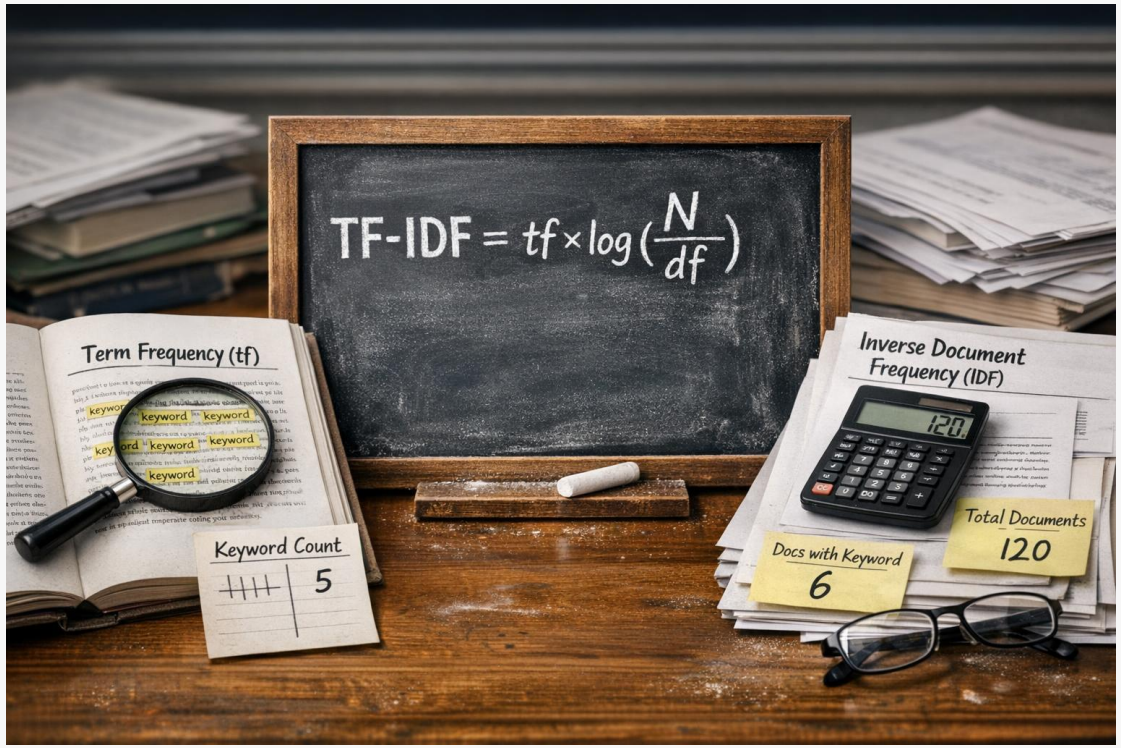
Reducción de términos comunes

IDF disminuye la relevancia de palabras comunes que aparecen en muchos documentos para mejorar la precisión.

Cálculo basado en logaritmo

IDF se calcula como el logaritmo del total de documentos dividido por los que contienen la palabra clave.

CÁLCULO Y FÓRMULA DE TF-IDF



Cálculo de TF-IDF

TF-IDF se calcula multiplicando la frecuencia del término por la frecuencia inversa del documento.

Importancia de TF-IDF

Resalta términos frecuentes en un documento pero raros en la colección total para mejorar búsquedas.

Aplicación en búsqueda

Mejora la precisión en la clasificación y resultados de búsqueda mediante la fórmula TF-IDF.

CÁLCULO Y FÓRMULA DE TF-IDF

1 Count Documents



Pizza — Appears in 5 out of 100 documents

DF = 5/100 `0.05`



The — Appears in all 100 documents

DF = 100/100 `1.0`

2 Flip to reward rare words



Pizza — High Score

IDF = 1/0.05 `20`



The — Low Score

DF = 1/1.0 `1.0`

3 Apply log



Pizza — Still higher

log(1/20 **)** `1.30`



The — Too common, no weight

log(1 **)** `0`

CÁLCULO Y FÓRMULA DE TF-IDF

Score = TF(word, doc) × log(Total docs / Docs containing word)

TF: Mide qué tan seguido aparece la palabra en un documento específico. Si un documento menciona "IA" 10 veces, es más probable que trate sobre ese tema que uno que solo la menciona 1 vez.
Ejemplo: En el Documento A (de 100 palabras), la palabra "IA" aparece 3 veces. **Su TF es 3 / 100 = 0.03.**

IDF: Mide qué tan "especial" o rara es la palabra en toda la biblioteca.
Ejemplo: Palabras como "el", "de" o "que" aparecen en todos los documentos (IDF bajo). Palabras como "algoritmo" o "redes neuronales" son más raras y nos ayudan a filtrar mejor (IDF alto).
Formula: log((Total de documentos) /(Documentos con la palabra).

CÁLCULO Y FÓRMULA DE TF-IDF

Ejemplo

Si tenemos **5 documentos** en total y la palabra "IA" aparece en **3 de ellos,** el cálculo del **IDF** sería:

IDF = log(total documentos / documentos en los que aparece el término) = log (5 / 3) = 0,22

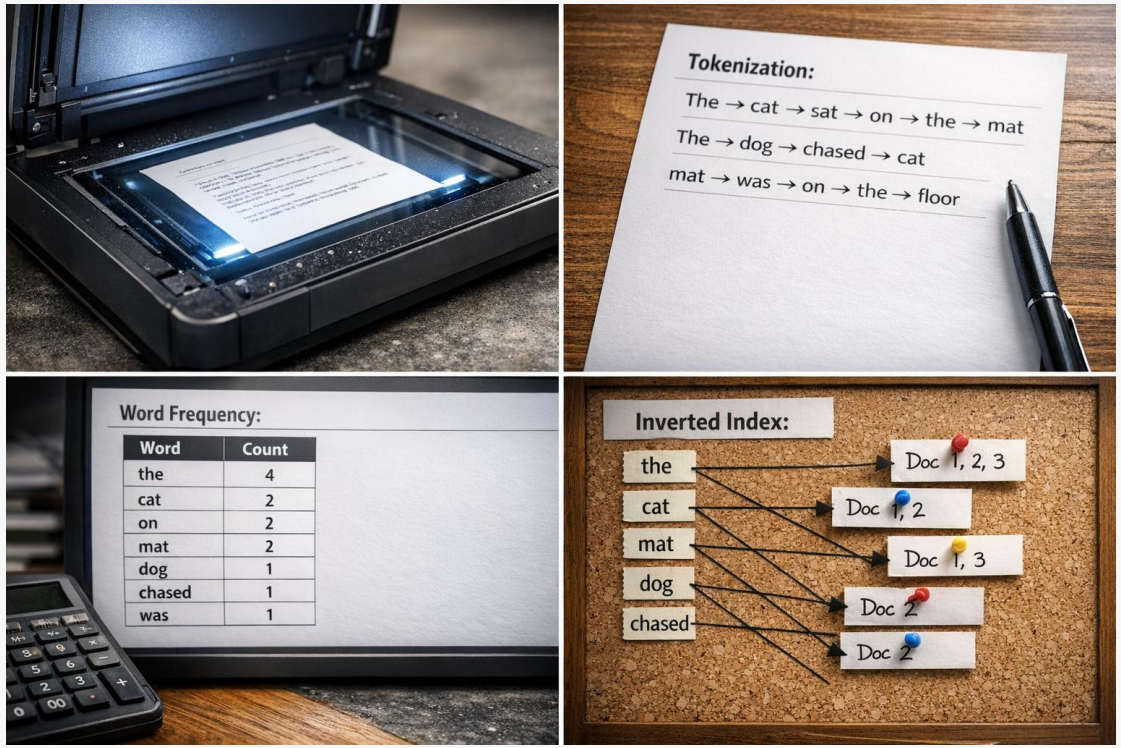
Ahora para obtener el TF-IDF final, necesitamos el valor del TF de un documento específico. Supongamos que en el Documento 1, la palabra "IA" aparece 3 veces y el texto tiene un total de 10 palabras.

TF = numero de veces / total palabras = 3 /10 = 0,3

TF-IDF (doc1) = TF(IA, doc1) * log (total documentos / docs (IA) = 0,3 * 0,22 = 0,066

APLICACIÓN DEL MECANISMO TF-IDF EN MOTORES DE BÚSQUEDA

PROCESO DE INDEXACIÓN Y ANÁLISIS DE DOCUMENTOS



Extracción de términos

El proceso inicia con la extracción de términos clave de los documentos para su análisis posterior.

Cálculo de frecuencias

Se calcula la frecuencia de cada término para evaluar su importancia relativa en los documentos.

Índice invertido

Los términos y sus frecuencias se almacenan en un índice invertido para facilitar búsquedas rápidas.

Preprocesamiento de texto

Aplicación de tokenización, eliminación de stopwords y stemming para mejorar la calidad de los datos.

RELEVANCIA Y CLASIFICACIÓN DE RESULTADOS

Uso de valores TF-IDF

Los valores TF-IDF asignan puntajes que indican la importancia de términos en documentos para una consulta específica.

Ordenación por relevancia

Los documentos se ordenan para mostrar primero los más relevantes según los puntajes calculados, mejorando la búsqueda.

Mejora de experiencia usuario

Reducir documentos irrelevantes incrementa la precisión y velocidad en la recuperación de información para el usuario.



EJEMPLOS PRÁCTICOS DE BÚSQUEDA CON TF-IDF



Búsqueda basada en relevancia

TF-IDF prioriza documentos con términos menos comunes pero altamente relevantes para mejorar resultados de búsqueda.

Diferencia con búsqueda simple

A diferencia de la coincidencia básica, TF-IDF valora términos especializados para un mejor posicionamiento.

Ejemplo de inteligencia artificial

En búsquedas de inteligencia artificial, términos como aprendizaje profundo reciben mayor importancia con TF-IDF.

VENTAJAS, DESAFÍOS Y EVOLUCIÓN DE TF-IDF



BENEFICIOS FRENTE A MÉTODOS TRADICIONALES

Relevancia local y global

TF-IDF diferencia términos evaluando su importancia tanto en documentos individuales como en toda la colección, mejorando la búsqueda.

Mejora de precisión

Al ponderar términos relevantes, TF-IDF aumenta la precisión y la utilidad de los resultados de búsqueda frente a métodos tradicionales.

Estándar duradero

La eficacia de TF-IDF ha sostenido su uso como estándar en recuperación de información durante varias décadas.



LIMITACIONES Y RETOS EN GRANDES VOLÚMENES DE DATOS

Limitaciones semánticas

TF-IDF no captura relaciones semánticas profundas ni maneja bien términos con múltiples significados.

Desafíos en big data

El procesamiento de grandes volúmenes puede comprometer la eficiencia de TF-IDF sin optimizaciones adecuadas.

Necesidad de modelos avanzados

Se requieren modelos complementarios para recuperar información contextualizada y mejorar resultados.

INTEGRACIÓN CON MODELOS AVANZADOS Y TENDENCIAS ACTUALES



Integración con Aprendizaje Automático

TF-IDF se combina con modelos de aprendizaje automático para mejorar la comprensión profunda del lenguaje.

Uso de Redes Neuronales

Las redes neuronales potencian la semántica contextual complementando la eficiencia de TF-IDF en sistemas avanzados.

Sistemas de Embeddings y Transformers

La integración de TF-IDF con embeddings y transformers amplía la precisión y aplicabilidad en procesamiento del lenguaje.

Mejora de la Experiencia de Usuario

Estas tendencias tecnológicas mejoran la interacción y resultados en sistemas basados en texto para el usuario final.

CONCLUSIÓN

Mejora en relevancia y precisión

TF-IDF transforma la búsqueda mejorando la relevancia y precisión en los resultados de palabras clave.

Desafíos con grandes datos

Existen desafíos al aplicar TF-IDF en grandes volúmenes de datos que requieren soluciones especializadas.

Integración con tecnologías avanzadas

La integración de TF-IDF con tecnologías avanzadas promete un futuro mejor en recuperación de información.

EJEMPLO

Código Python

EJEMPLO

```python
import numpy as np

# 1. Nuestros 5 documentos
documentos = [
    "la ia aprende de datos",
    "los datos son el futuro",
    "la ia y el futuro de la tecnologia",
    "aprender sobre tecnologia es vital",
    "el futuro es hoy"
]
```

Listado de documentos

```python
# 2. Parámetros de búsqueda
termino_busqueda = "futuro"
N = len(documentos) # Total de documentos = 5
```

Cantidad de documentos

```python
# 3. Cálculo del IDF (Global)
# ¿En cuántos documentos aparece "futuro"?
df = sum(1 for doc in documentos if termino_busqueda in doc)
idf = np.log10(N / df)
```

Cálculo del IDF general

EJEMPLO

```python
# 4. Cálculo del TF-IDF para cada documento
scores = []
for i, doc in enumerate(documentos):
    palabras = doc.split()
    total_palabras = len(palabras)

    # Contamos cuántas veces aparece el término en ESTE documento
    veces_aparece = palabras.count(termino_busqueda)

    # TF = Frecuencia local
    tf = veces_aparece / total_palabras

    # Score final = TF * IDF
    score_final = tf * idf
    scores.append((i + 1, score_final, doc))
```

Cálculo del TF de cada documento y el TF-IDF para cada uno

```python
# 5. Ordenamos por el Score más alto
scores.sort(key=lambda x: x[1], reverse=True)
```

```python
print("Ranking de relevancia:")
for rank, (doc_id, score, texto) in enumerate(scores, 1):
    print(f"{rank}. Doc {doc_id} (Score: {score:.4f}): {texto}")
```

```
Ranking de relevancia:
1. Doc 5 (Score: 0.0555): el futuro es hoy …
3. Doc 3 (Score: 0.0277): la ia y el futuro de la tecnologia
4. Doc 1 (Score: 0.0000): la ia aprende de datos
5. Doc 4 (Score: 0.0000): aprender sobre tecnologia es vital
```

Ordenamiento y presentación de los resultados